NextGenQA

Intelligent Test Case Generation

Research Project Checklist

Component overview · system architecture · novelty layers · repository map · deliverables checklist

GITHUB REPOSITORY

`github.com/R26-SE-039/Intelligent-Test-Case-Generation`

SUBMITTED BY

ABEYGUNASEKARA D T

Student ID: IT22303684

Project R26-SE-039 · Final-year research

Prepared for PP1 viva & supervisor review

1. Project Overview

NextGenQA is a web application that helps QA engineers automatically generate test cases from user stories. It converts plain-English user stories into Gherkin scenarios, then into runnable test code for Selenium, Playwright, or Cypress — and finally executes the tests on a clean cloud runner. Component 2 also adds **Agent Explorer**, an autonomous AI agent that discovers test scenarios from a single QA intent.

Repository map

Path	What lives here	Stack
`/backend`	FastAPI service, agent loop, DOM crawler, Jinja2 code generators. open	Python 3.11 · FastAPI · Playwright
`/frontend`	Next.js 16 dashboard (eight pages, live WebSocket streams). open	TypeScript · React 19 · Tailwind 4
`/scripts`	Dev/setup scripts & one-off utilities. open	Bash · Python
`NextGenQA_Component2_Guide.md`	Component 2 master guide. open	Markdown
`docker-compose.yml`	Container orchestration for local dev	Docker

How to read this document

Each section is a self-contained checklist card. Section 2 shows the end-to-end pipeline diagram. Sections 3.1 – 3.8 walk through every page of the application with a short summary, key features, and the relevant tech. Section 4 covers the novelty layers of Agent Explorer. Section 5 lists the project-level outcomes and deliverables ticked off so far.

2. System Architecture

The application is a linear seven-stage pipeline. The QA engineer can either follow the standard route — Projects → User Story → Gherkin → Mode → DOM → Code Review → Execution — or use the autonomous **Agent Explorer** entry, which collapses the entire pipeline into a single intent sentence.

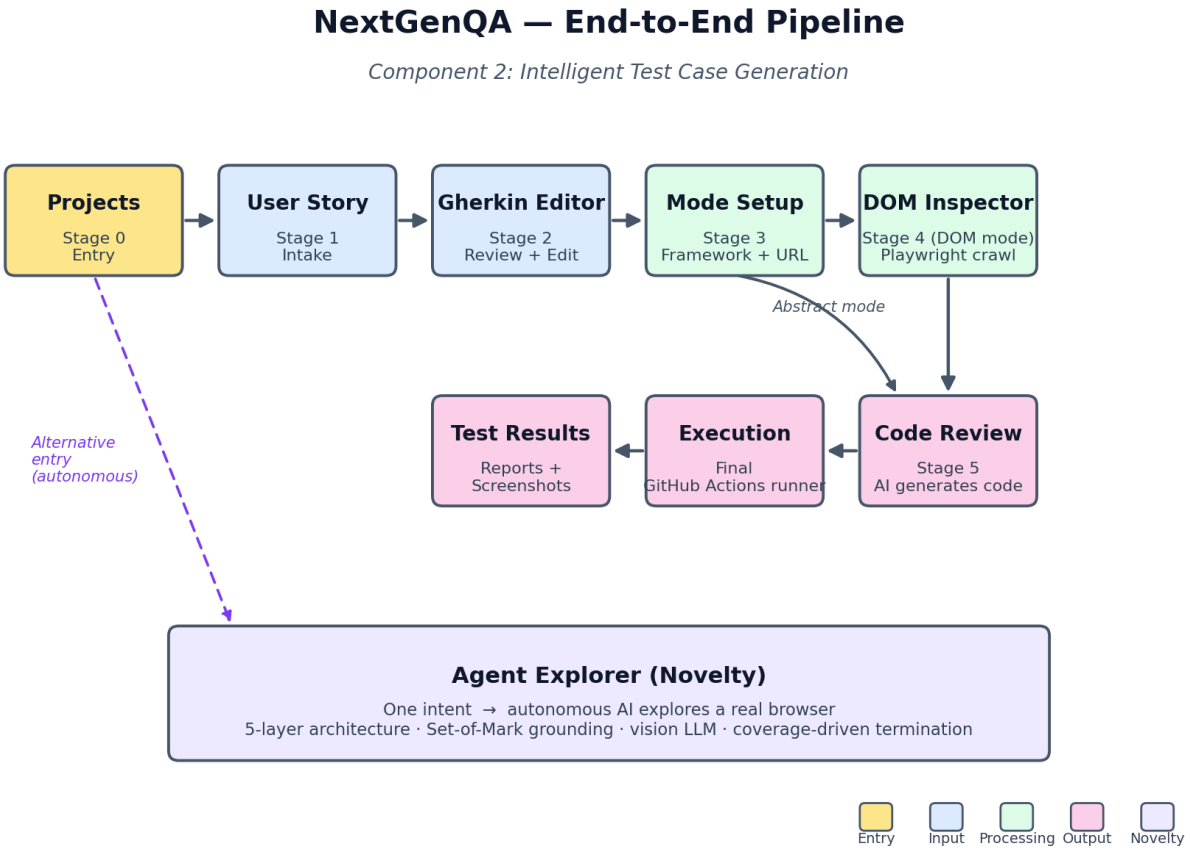


Figure 1. End-to-end pipeline. Top row is the linear flow through the eight pages. Bottom row shows the AI-generated code being executed and the resulting test report. The purple box is the research contribution (Component 2 novelty).

3. Component Checklist

Each card below corresponds to one page of the application. The accent strip shows the pipeline stage. The colour indicates which part of the flow the page belongs to: amber = entry, blue = input, green = processing, pink = output, purple = research novelty.

STAGE 0	Projects Page
Summary	First page the user lands on. Each project is an isolated workspace — separate user stories, Gherkin scenarios, DOM elements, and test results.
Key features	• Create / open / delete projects • Per-project story count and creation date • Empty state with first-project CTA • Multi-project isolation backed by PostgreSQL
Tech / where	Frontend: `/frontend/src/app/projects` · Backend: `/backend/app/projects`

STAGE 1	User Story Intake
Summary	First stage of the pipeline. The QA adds user stories — either manually (Actor / Action / Goal form) or imported from Component 1 of the research project.
Key features	• Dashboard stats: total / pending / processing / completed • Priority filter (High / Medium / Low) and bulk select • Bridge button *Generate Gherkin* calls the LLM • Stories persist immediately with status tracking
Tech / where	Frontend: `/frontend/src/app/dashboard` · LLM: Anthropic Claude (text)

STAGE 2	Gherkin Editor
Summary	AI converts each user story into Given / When / Then format. The QA reviews, edits, and approves each scenario. Only approved Gherkin feeds the next stage.
Key features	• Monaco editor (same engine as VS Code) with syntax highlighting • Per-story Save · Regenerate · Approve flow • Copy-to-clipboard support • Approval state stored in PostgreSQL
Tech / where	Frontend: `/frontend/src/app/dashboard/gherkin-editor` · Backend: Jinja2 prompts in `/backend`

STAGE 3 — Mode Setup

Summary

Configures how the test code is generated. Three choices: test mode (Abstract vs DOM), framework (Selenium / Playwright / Cypress), and target URL.

Key features

- Abstract vs DOM mode toggle
- Framework selector — Selenium · Playwright · Cypress
- URL probe button: returns HTTP status & page title
- Smart routing: DOM → Inspector · Abstract → Code Review

Tech / where

Frontend: `/frontend/src/app/dashboard/mode-setup` · Backend: URL probe endpoint

STAGE 4 — DOM Inspector (DOM mode only)

Summary

Crawls the target website with a real Playwright browser, captures all interactive elements, and stores the real CSS selectors for code generation.

Key features

- Real Chromium crawling via Playwright
- Live log streaming over Server-Sent Events
- Authenticated crawling (login form selectors)
- QA can edit / add / delete / bulk-approve elements

Tech / where

Backend: `/backend/app/dom_crawler` · Frontend: `/frontend/.../dom-inspector`

STAGE 5 — Code Review

Summary

AI combines approved Gherkin + DOM elements + framework to produce ready-to-run test code. Three tabs show the same logic in Selenium, Playwright, and Cypress.

Key features

- Three-tab framework switcher with own filenames
- Monaco editor — read & edit AI output directly
- Buttons: Generate · Save · Copy · Download · Run
- Stale-input warning when Gherkin / DOM changes

Tech / where

Backend: Jinja2 generators per framework · Frontend: `/frontend/.../code-review`

FINAL — Execution

Summary

Runs the generated tests on a clean GitHub Actions runner and streams the live logs back to the dashboard. Closes the loop from user story to test result.

Key features

- Cloud execution via GitHub Actions
- Live log view with timestamps and status icons
- Recharts pie chart of pass / fail summary
- Screenshot gallery per scenario
- Allure HTML report download

Tech / where

Backend: GitHub Actions dispatcher · Frontend: Recharts + WebSocket log viewer

NOVELTY	Agent Explorer (research contribution)
Summary	One short intent sentence — e.g. ‘check the login flow’ — triggers an autonomous AI agent that opens a real browser, decides what to test, executes actions, and reports back the scenarios it discovered.
Key features	• Three-panel UI: Live View · Reasoning · Coverage & Discoveries • Headed Chromium for visual demo • Set-of-Mark numbered overlay (vision grounding) • Four role-tagged thoughts per step (Planner / Actor / Observer / Critic) • Coverage-driven termination — agent stops when novelty plateaus • Reflexion memory — lessons from failed steps fed back in
Tech / where	Backend: `/backend/app/agent_explorer` · Frontend: `/frontend/.../agent-explorer`

4. Research Novelty — Five Layers

The research contribution is not the LLM itself — that is a commodity. The contribution is the five-layer architecture stitched together for the QA test-exploration domain. Each layer is grounded in a published 2022–2025 research technique, and Layer 5 is our novel addition.

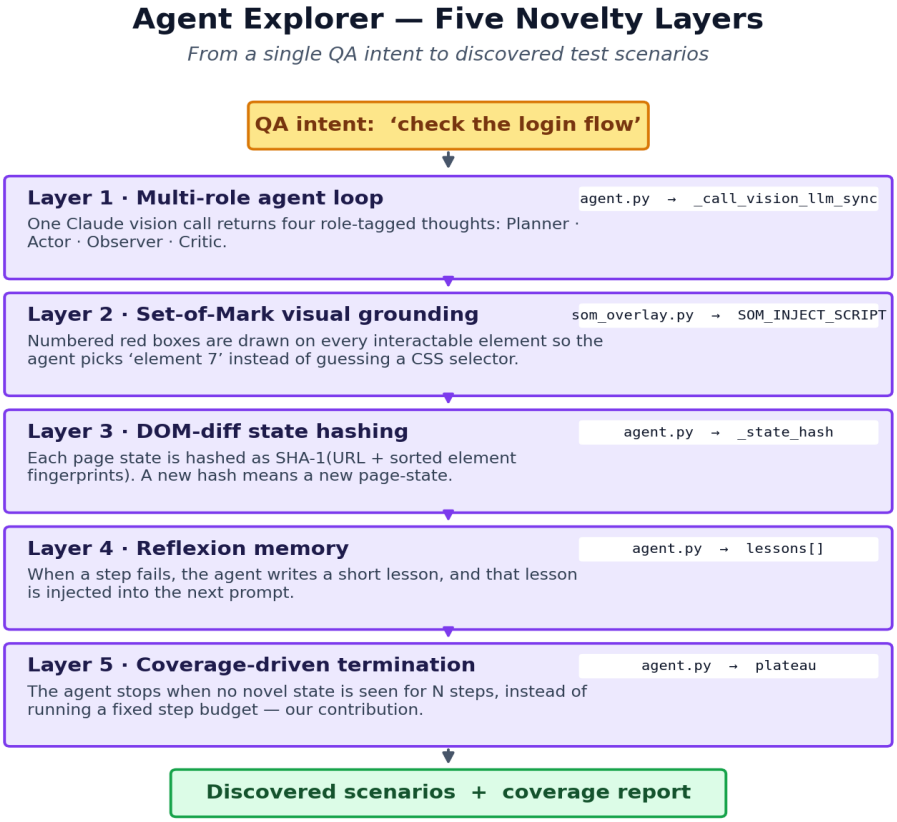


Figure 2. The five-layer architecture of Agent Explorer, with the file and function inside `/backend/app/agent_explorer` where each layer is implemented.

4.1 Layers & literature mapping

#	Layer	Source
1	Multi-role agent loop — Planner · Actor · Observer · Critic, all in one LLM call	ReAct — Yao et al. 2022
2	Set-of-Mark visual grounding — numbered red boxes on screenshots	Yang et al. 2023
3	DOM-diff state hashing — SHA-1 over URL + sorted element fingerprints	Inspired by WebArena 2024
4	Reflexion memory — failed-step lessons re-injected into the next prompt	Shinn et al. 2023 (Reflexion)
5	Coverage-driven termination — stop when no novel state for N steps	Our novel contribution

4.2 Defence one-liner

> *“Without this five-layer stack, the LLM would hallucinate CSS selectors, run forever, and never know what it has already covered. The contribution is integrating these published techniques into a single agent specifically for QA test exploration — and adding coverage-driven termination as the novel fifth layer.”*

5. Technology Stack

Stack summary across the three tiers. The repository is roughly 33 % TypeScript, 27 % Python, and 37 % Jupyter notebooks (research experiments).

5.1 Backend

Layer	Choice
Framework	FastAPI · async + WebSocket
LLM	Anthropic Claude Sonnet 4.6 (vision)
Browser automation	Playwright (sync) — headed for demo, headless in CI
DB / persistence	PostgreSQL · SQLAlchemy
Pub/sub	In-memory AgentBroker per run (swappable for Redis)
Code generation	Jinja2 templates per framework
Hashing	hashlib.SHA-1 for state-hash novelty detection
Validation	Pydantic models for typed request / response

5.2 Frontend

Layer	Choice
Framework	Next.js 16 · React 19 · App Router
Language	TypeScript (strict mode, no any)
Styling	TailwindCSS 4
Code editor	Monaco (the VS Code editor)
Charts	Recharts (test-result pie chart)
Icons	lucide-react
Streaming	Native WebSocket + Server-Sent Events
State	React 19 hooks · TypeScript discriminated unions for events

5.3 Infrastructure & tooling

Layer	Choice
Containers	Docker · docker-compose for local dev
CI / execution	GitHub Actions runners for test execution
Test reports	Allure HTML
Build	Makefile · pnpm · pip

6. Outcomes & Deliverables Checklist

Status of the deliverables for the PP1 milestone. Green-check items are complete and demonstrable in the live demo; open-square items are planned for PP2 or currently in progress.

Functional deliverables

- [x] Multi-project workspace with isolated user stories & results
- [x] User Story Intake page with manual entry & Component-1 import
- [x] AI Gherkin generation with Monaco editor and per-story approval
- [x] Mode Setup with URL probe and Abstract / DOM branching
- [x] DOM Inspector — Playwright crawler with live log streaming
- [x] Three-framework code generation (Selenium, Playwright, Cypress)
- [ ] Cloud execution via GitHub Actions with screenshot gallery & Allure (planned — not built yet)
- [x] Agent Explorer end-to-end: intent → SoM browser → discovered scenarios

Research deliverables

- [x] Five-layer agent architecture implemented and documented
- [x] Set-of-Mark overlay injector working on live Chromium
- [x] DOM-diff state hashing producing reproducible state IDs
- [x] Reflexion memory loop demonstrated on failed selectors
- [x] Coverage-driven termination — agent stops on novelty plateau
- [ ] Quantitative evaluation: coverage vs fixed-budget agents
- [ ] Comparison study: ours vs baseline LLM-only agent

Engineering deliverables

- [x] FastAPI backend with typed Pydantic models & WebSocket streams
- [x] Next.js 16 / React 19 frontend with TypeScript discriminated unions
- [x] Containerised via Docker Compose for one-command local run
- [x] Makefile for unified dev / build / test workflows
- [ ] End-to-end CI pipeline (lint → test → deploy preview)

Documentation

- [x] Component 2 master guide — NextGenQA_Component2_Guide.md
- [x] Per-page speech notes (local, to be added to repo)
- [x] Architecture diagrams (this PDF + local Mermaid sources)
- [ ] Final dissertation chapter on agent novelty (in progress)

7. Quick Links

All URLs in this document, collected in one place for easy reference.

Resource	URL
GitHub repository (root)	https://github.com/R26-SE-039/Intelligent-Test-Case-Generation
Backend source	https://github.com/R26-SE-039/Intelligent-Test-Case-Generation/tree/main/backend
Frontend source	https://github.com/R26-SE-039/Intelligent-Test-Case-Generation/tree/main/frontend
Scripts	https://github.com/R26-SE-039/Intelligent-Test-Case-Generation/tree/main/scripts
Component 2 master guide	https://github.com/R26-SE-039/Intelligent-Test-Case-Generation/blob/main/NextGenQA_Component2_Guide.md